

Software Quality Assurance

Lecture 10

Outline

- Role of specification
- Levels of specifications vs levels of testing
- SW testing strategies: bigbang vs incremental
- Top down testing vs bottom up testing (Stubs vs drivers)
- SW test classifications: Blackbox vs whitebox

Role of specification

- Validation and verification activities, such as testing, cannot be meaningful unless we have a specification for the software
- The software we are building could be a single module or class, or could be an entire system
- Depending on the size of the project and the development methods, specifications can range from a single page to a complex hierarchy of interrelated documents

Levels of specifications vs levels of testing

There are usually at least three levels of software specification documents in large systems:

1. Functional specifications (or requirements) give a precise description of the required behaviour of the system – *what the software should do, not how it should do it – may also describe constraints on how this can be achieved*
2. Design specifications describe the architecture of the design to implement the functional specification – *the components of the software and how they are to relate to one another*
3. Detailed design specifications describe how each component of the architecture is to be implemented – *down to the individual code units*

Given the hierarchy of specifications, it is usual to structure testing into **three** (or more) corresponding levels:

3. Unit testing addresses the verification that individual components meet their detailed design specification
2. Integration testing verifies that the groups of units corresponding to architectural elements of the design specification can be integrated to work as a whole
1. System testing verifies that the integrated total product has the functionality specified in the functional specification

To these levels it is usual to add the additional test level:

0. Acceptance testing, in which the actual customers validate that the software meets their real intentions as well as what has been functionally specified, and accept the result

SW testing strategies: bigbang vs incremental

- To test the software in its entirety, once the completed package is available; known as **“big bang testing”**
- To test the software piecemeal, in modules, as they are completed **(unit tests)**;
- then to test groups of tested modules integrated with newly completed modules **(integration tests)**
- This process continues until all the package modules have been tested. Once this phase is completed, the entire package is tested as a whole **(system test)**
- This testing strategy is usually termed **“incremental testing”**

Top down testing vs bottom up testing (Stubs vs drivers)

- incremental testing is also performed according to two basic strategies: bottom-up and top-down.
- In top-down testing, the first module tested is the main module, the highest level module in the software structure; the last modules to be tested are the lowest level modules. (stubs)
- In bottom-up testing, the order of testing is reversed: the lowest level modules are tested first, with the main module tested last. (drivers)

Stubs and drivers are two such elements used in software testing process, which act as a temporary replacement for a module. These are an integral part of software testing process as well as general software development.

Stubs vs drivers

Consider an example of a web application, which consists of 4 modules i.e., Module-A, Module-B, Module-C and Module-D. Each of the following modules is responsible for some specific activity or functionality, as under

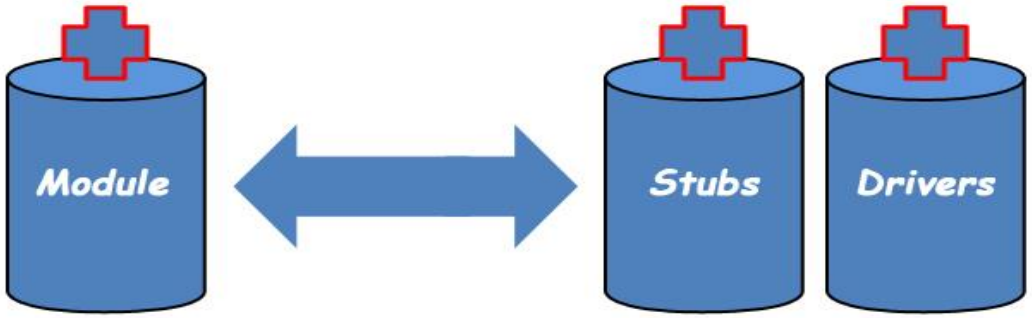
Module-A ? Login page of the web application.

Module-B → Home page of the web application.

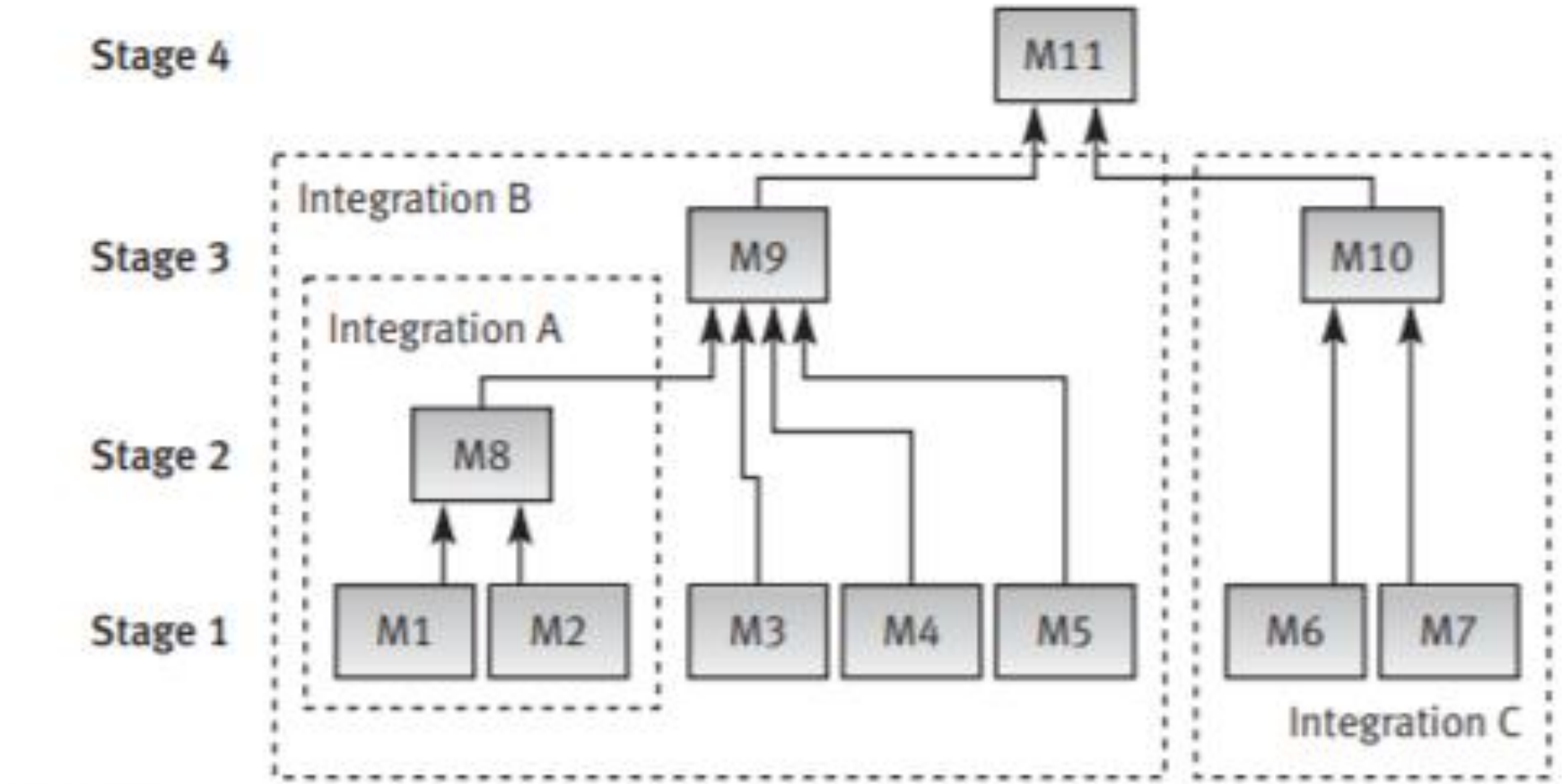
Module-C → Print Setup.

Module-D → Log out page.

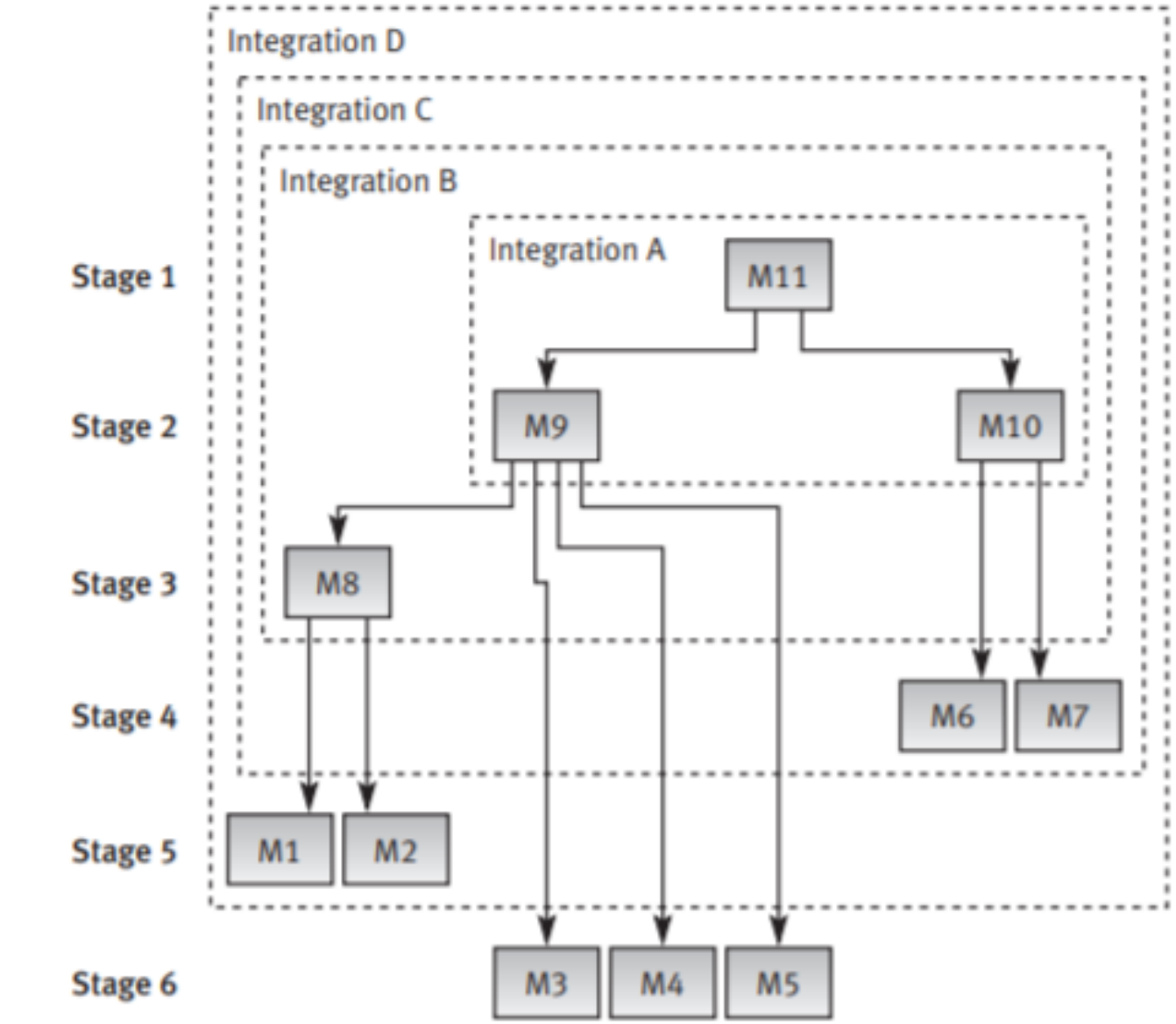
modules A, B, C & D involves the interdependencies of each module over other.



It is always preferable, to perform testing, in parallel, to the development process



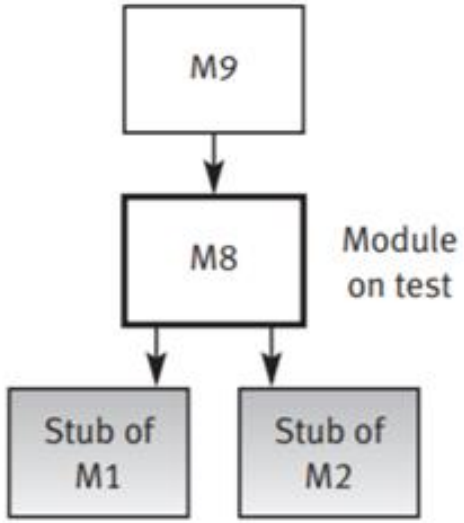
(a) Bottom-up testing

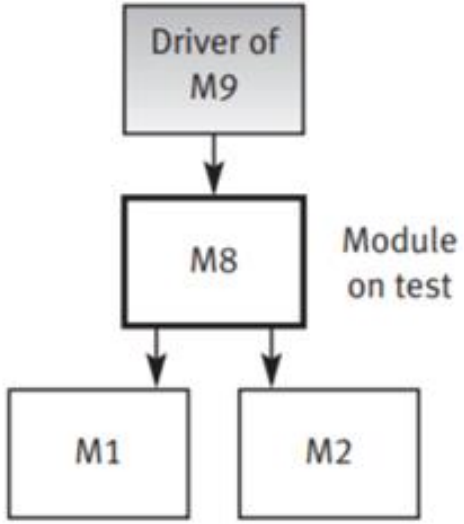


(b) Top-down testing

(a) Implementing top-down tests

(b) Implementing bottom-up tests





- Stubs are commonly referred to as "called programs" and are being used in top bottom approach of the integration testing, whereas drivers are "calling program" and they are used in bottom-up integration testing.

User Authentication Module:

Stub: Imagine the authentication module isn't fully implemented yet, but other modules need to interact with it. We can create a stub for the authentication module that mimics its behavior. For example, the stub could return a predefined user ID and indicate successful authentication regardless of the input credentials.

Driver: If we're testing a module that depends on user authentication, such as accessing user-specific order history, we might create a driver to simulate interactions with the authentication module. The driver would provide the necessary inputs to trigger authentication-related functionality.

Pros and cons of incremental testing

- In incremental testing, a great part of the errors are identified and corrected at an earlier stage of development and testing, which prevents “migration” of escaped defects to a later, more complex stage in the development where their correction would require significantly greater resources.
- The main disadvantage of incremental testing is the quantity of programming resources required for preparation of stubs and drivers for the unit and integration tests. Another major disadvantage is the need to carry out numerous testing operations for the same program (big bang testing requires only a single testing operation).
- It is generally accepted that incremental testing should be preferred despite its disadvantages.

SW test classifications: Blackbox vs whitebox

- Black box (functionality) testing. Identifies bugs only according to software malfunctioning as they are revealed in its erroneous outputs. In cases that the outputs are found to be correct, black box testing disregards the internal path of calculations and processing performed.
    - Testing that ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions
- White box (structural) testing. Examines internal calculation paths in order to identify bugs. Although the term “white” is meant to emphasize the contrast between this method and black box testing, the method’s other name – “glass box testing” – better expresses its basic characteristic, that of investigating the correctness of code structure.
    - Testing that takes into account the internal mechanism of a system or component

WHITE BOX TESTING

Different paths in a software module are created by the choice in conditional statements, such as IF–THEN–ELSE or DO WHILE or DO UNTIL. Path testing is motivated by the aspiration to achieve complete coverage of a program by testing all its possible paths.

■ “Path coverage” – to plan our test to cover all the possible paths, where coverage is measured by percentage of paths covered.

it is impractical in most cases because of the vast resources required for its performance. Just how costly these applications can be is illustrated in the following example. Let us now calculate the number of possible paths created by a simple module containing 10 conditional statements, each allowing for only two options (e.g., IF–THEN–ALSO and DO WHILE). This simple module contains 1024 different paths.

In other words, in order to obtain full path coverage for this module (probably 25–50 lines of code) one should prepare at least 1024 test cases, one for each possible path. A straightforward calculation of the number of test cases required to test a software package that contains 100 modules of similar complexity (a total of 102 400 test cases) readily indicates the impracticality of wide use of path testing.

Hence, its application is directed mainly to high risk software modules, where the costs of failure resulting from software error fully warrant the costs of path testing. This situation has encouraged development of an alternative yet weaker coverage concept – line coverage

The code for the program would look something like this:

```
input x
if x > 5 then
return A
else return B
```

In order for path coverage testing to effectively "cover all paths," the two test cases must be run, with x greater than 5 and x less than or equal to 5.

“Line coverage” – to plan our tests to cover all the program code lines, where coverage is measured by percentage of lines covered. The line coverage concept requires that, for full line coverage, every line of code be executed at least once during the process of testing.

```
if(cond) {
    line1();
    line2();
    line3();
    line4();
} else {
    line5();
}
```

If your test only exercises the cond being true and never runs the else branch you have:

- 4 out of 5 lines covered
- 1 out of 2 branches covered

Consider code:

READ X
READ Y
I F X>Y THEN Z = 0
ENDIF

To achieve 100% statement coverage of this code segment just one test case is required, one which ensures that variable A contains a value that is greater than the value of variable Y, for example, X = 12 and Y = 10.

In the source code, when each occurring condition is evaluated for both true and false states, then the **Condition Coverage** for the code is said to be complete.

```
Add (int a, int b)
{
    If (b > a)
    {
       b = b - a
       Print b
    }
    If (a > b)
    {
          b = a – b
          Print b
    }
       Else Print ‘0’
}
```

- **For example,** in the above code if value sets (2, 3) and (4, 2) are used then Condition Coverage would be 100%. When data set (2, 3) is used then (b > a) evaluates to true and (a > b) evaluates to false. Similarly, when data set (4, 2) is used then (b > a) evaluates to false and (a > b) evaluates to true.
- Thus both the conditions have both the values i.e true and false covered. Hence the Condition Coverage would be 100%.

Branch Coverage Vs Condition Coverage

Branch Coverage is often confused with Condition Coverage, however, the two are different.

Let's understand this with a simple example.

If (a >0) & (b >0)

Then Print "Hello"

Else Print "Bye"

Let us write down the data set needed for complete Branch Coverage:

(1, 1) – In this case, 'a' and 'b' both are true, so the If condition gets executed.
(1, 0) – In this case, 'a' is true and 'b' would be false, so the Else part of the code is executed.

As we know the purpose of Branch Coverage is to get every branch executed at least once and this purpose is achieved.

Condition Coverage:

(1, 0) – In this case, ‘a’ is true and ‘b’ would be false.
(0, 1) – In this case, ‘a’ is false and ‘b’ would be true.

The purpose of Condition Coverage is to get each of true and false for every condition executed and this purpose is achieved here.

```
Add (int a, int b)
{
    If (b > a)
    {
        b = b - a
        Print b
    }
    If (a > b)
    {
            b = a – b
            Print b
    }
        Else Print ‘0’
}
```

- **For example,** in the above code if value sets (2, 3), (4, 2), (1, 1) are used then Branch Coverage would be 100%. When data set (2, 3) is used then (b > a) and the first ‘If’ branch gets executed. Similarly, when data set (4, 2) is used then (a > b) evaluates to true and the second ‘If’ branch gets executed.
- Then with the data set (1, 1) the ‘Else’ branch evaluates to true and gets executed. Thereby, ensuring 100% Branch Coverage.

TOOLS for code coverage

- Parasoft JTest (java)
- Testwell CTC++
- Cobertura (java)
- JaCoCo (java)

Coverage.py is one of the most popular code coverage tools for Python. It uses code analysis tools and tracing hooks provided in Python standard library to measure coverage

Pytest-cov is a Python plugin to generate coverage reports

Advantages and disadvantages of white box testing

The main advantages of white box testing are:

■ Direct statement-by-statement checking of code enables determination of software correctness as expressed in the processing paths, including whether the algorithms were correctly defined and coded.

■ It ascertains quality of coding work and its adherence to coding standards.

The main disadvantages of white box testing are:

■ The vast resources utilized, much above those required for black box testing of the same software package.

■ The inability to test software performance in terms of availability (response time), reliability, load durability, and other testing classes related to operation, revision and transition factors. The characteristics of white box testing limit its use to software modules of very high risk and very high cost of failure, where it is highly important to identify and fully correct as many of the software errors as possible.

Black box testing

- Equivalence classes for output correctness tests
- Equivalence class partitioning is a black box method aimed at increasing the efficiency of testing and, at the same time, improving coverage of potential error conditions. An equivalence class (EC) is a set of input variable values that produce the same output results or that are processed identically.
- EC boundaries are defined by a single numeric or alphabetic value, a group of numeric or alphabetic values, a range of values, and so on. An EC that contains only valid states is defined as a “valid EC”, whereas an EC that contains only invalid states is defined as an “invalid EC”. In cases where a program’s input is provided by several variables, valid and invalid ECs should be defined for each variable.
- According to the equivalence class partitioning method, test cases are defined so that each valid EC and each invalid EC are included in at least one test case. Test cases are defined separately for the valid and invalid ECs. In defining a test case for the valid ECs, we try to cover as many as possible “new” ECs

- Compared to the use of a random sample of test cases, equivalence classes save testing resources because they eliminate duplication of the test cases defined for each EC.
- Importantly, as the equivalence class method is a black box method, equivalence class partitioning is based on software specification documentation, not on the code.
- Systematic constructing of equivalence classes for a program’s input variables increases the coverage of possible valid and error conditions of input and thus improves the testing plan’s effectiveness.

Example – the Golden Splash Swimming Center The following example illustrates the definition of (valid and invalid) equivalence classes and the corresponding test case values. The software module in question calculates entrance ticket prices for the Golden Splash Swimming Center.

The Center's ticket price depends on four variables: day (weekday, weekend), visitor's status (OT = one time, M = member), entry hour (6.00–19.00, 19.01–24.00) and visitor's age (up to 16, 16.01–60, 60.01–120).

	Mon, Tue, Wed, Thurs, Fri				Sat, Sun			
Visitor's status	OT	OT	M	M	OT	OT	M	M
Entry hour	6.00–19.00	19.01–24.00	6.00–19.00	19.01–24.00	6.00–19.00	19.01–24.00	6.00–19.00	19.01–24.00
	Ticket prices – $							
Visitor's age								
0.0–16.00	**5.00**	**6.00**	**2.50**	**3.00**	**7.50**	**9.00**	**3.50**	**4.00**
16.01–60.00	**10.00**	**12.00**	**5.00**	**6.00**	**15.00**	**18.00**	**7.00**	**8.00**
60.01–120.00	**8.00**	**8.00**	**4.00**	**4.00**	**12.00**	**12.00**	**5.50**	**5.50**

A total of 15 test cases cover all the defined ECs

Variable	Valid equivalence classes	Representing values: Values for valid ECs	Representing values: Boundary values	Invalid equivalence classes	Representing values for invalid ECs
Day of week	(1) Mon, Tue, Wed, Thurs, Fri	Mon		(1) Any alpha-numeric value (not a day)	Mox
	(2) Sat, Sun	Sat			
Visitor's status	(1) OT (2) M	OT M		Other than OT or M	88
Entry hour	(1) 6.00–19.00 (2) 19.01–24.00	7.55 20.44	6.00, 19.00 19.01, 24.00	(1) Hours < 6.00 (2) Any alpha-numeric values (not time)	4.40 &@
Visitor's age	(1) 0.0–16.00 (2) 16.01–60.00 (3) 60.01–120.00	8.4 42.7 65.0	0.0, 16.00 16.01, 60.00 60.01, 120.00	(1) Any alpha-numeric value (not an age) (2) Ages > 120.0	TTR 150.1

Test case type	Test case no.	Day of week	Visitor's status	Entry hour	Visitor's age	Test case results
For valid ECs	1	Mon	OT	7.55	8.4	$5.00
	2	Sat	M	20.44	42.7	$8.00
	3	Sat	M	22.44	65.0	$5.50
	4	Sat	M	6.00	0.0	$3.50
	5	Sat	M	19.00	16.00	$3.50
	6	Sat	M	19.01	16.01	$8.00
	7	Sat	M	19.01	60.00	$8.00
	8	Sat	M	24.00	60.01	$5.50
	9	Sat	M	24.00	120.0	$5.50
For invalid ECs	10	Mox	OT	7.55	8.4	Invalid day
	11	Mon	88	7.55	8.4	Invalid visitor status
	12	Mon	OT	4.40	8.4	Invalid entry hour
	13	Mon	OT	&@	8.4	Invalid entry hour
	14	Mon	OT	7.55	TTR	Invalid visitor age
	15	Mon	OT	7.55	150.1	Invalid visitor age

Advantages and disadvantages of black box testing

The main advantages of black box testing are:

■ Black box testing allows us to carry out the majority of testing classes, most of which can be implemented solely by black box tests. Of the test classes unique to black box testing, of special importance are system performance tests such as load tests and availability tests.

■ For testing classes that can be carried out by both white and black box tests, black box testing requires fewer resources than those required for white box testing of the same software package. The main disadvantages of black box testing are:

■ Possibility that coincidental aggregation of several errors will produce the correct response for a test case, and prevent error detection. In other words, black box tests do not readily identify cases of errors that counteract each other to accidentally produce the correct output.

■ Absence of control of line coverage. In cases where black box testers wish to improve line coverage, there is no easy way to specify the parameters of the test cases required to improve coverage. Consequently, black box tests may not execute a substantial proportion of the code lines, which are not covered by a set of test cases.

■ Impossibility of testing the quality of coding and its strict adherence to the coding standards.

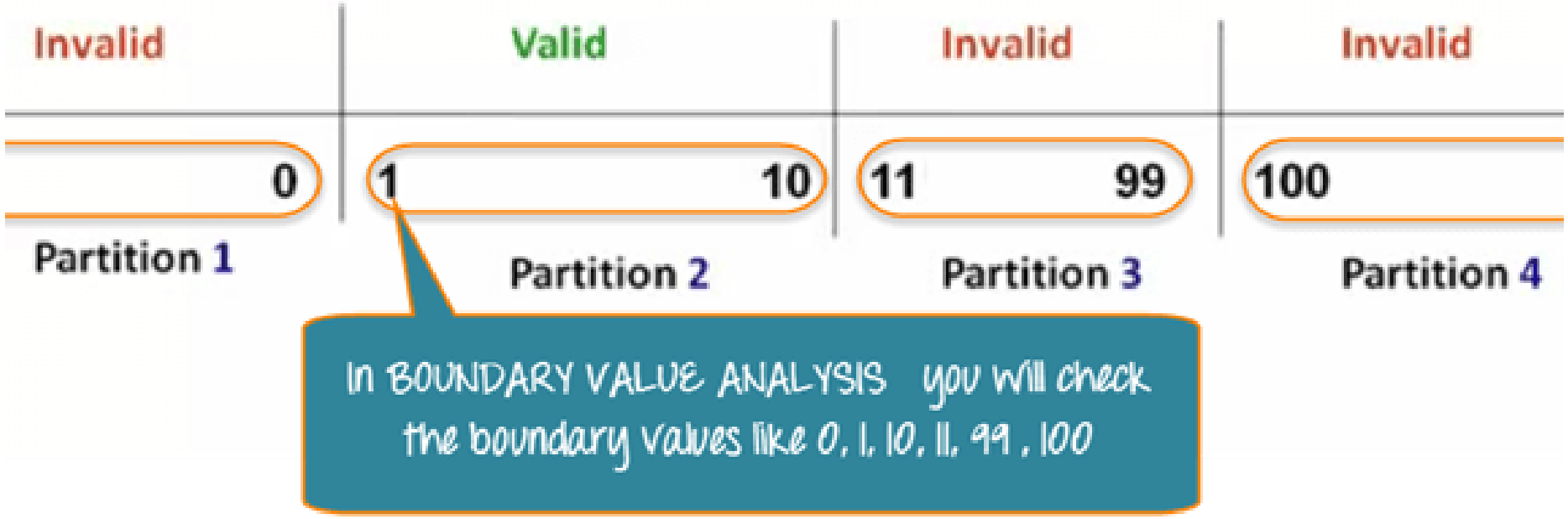
What is Boundary Testing?

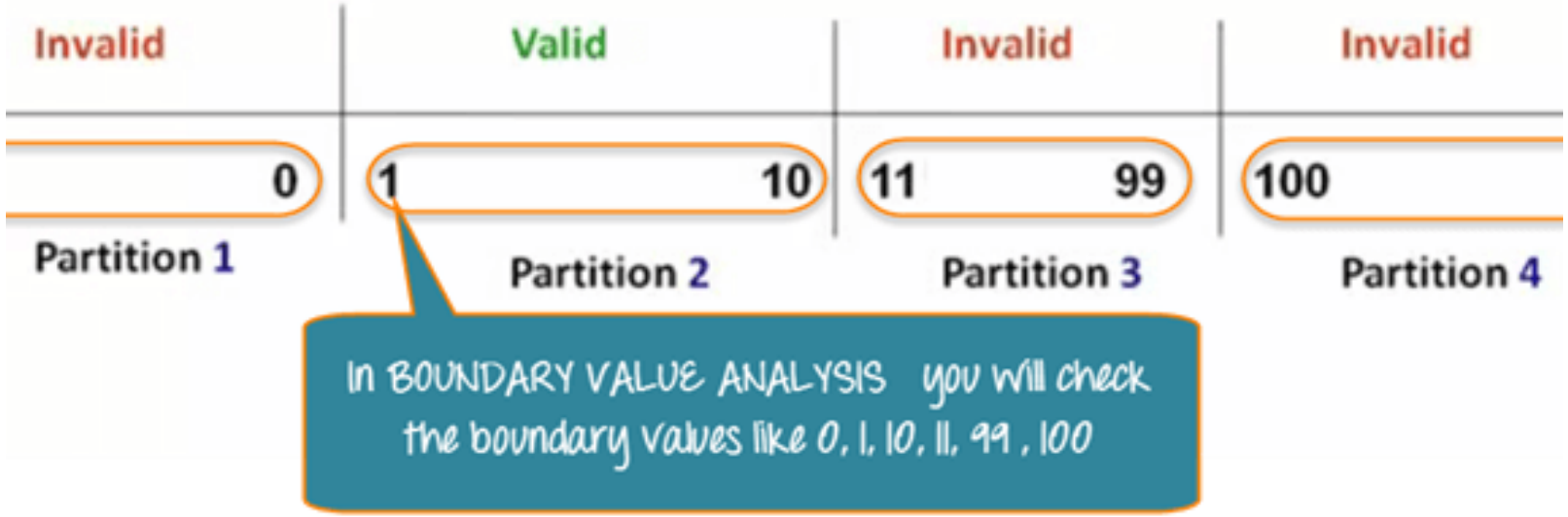
Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

1. Minimum
2. Just above the minimum
3. A nominal value
4. Just below the maximum
5. Maximum

Boundary Testing comes after the Equivalence Class Partitioning.

Boundary Value Analysis- in Boundary Value Analysis, you test boundaries between equivalence partitions





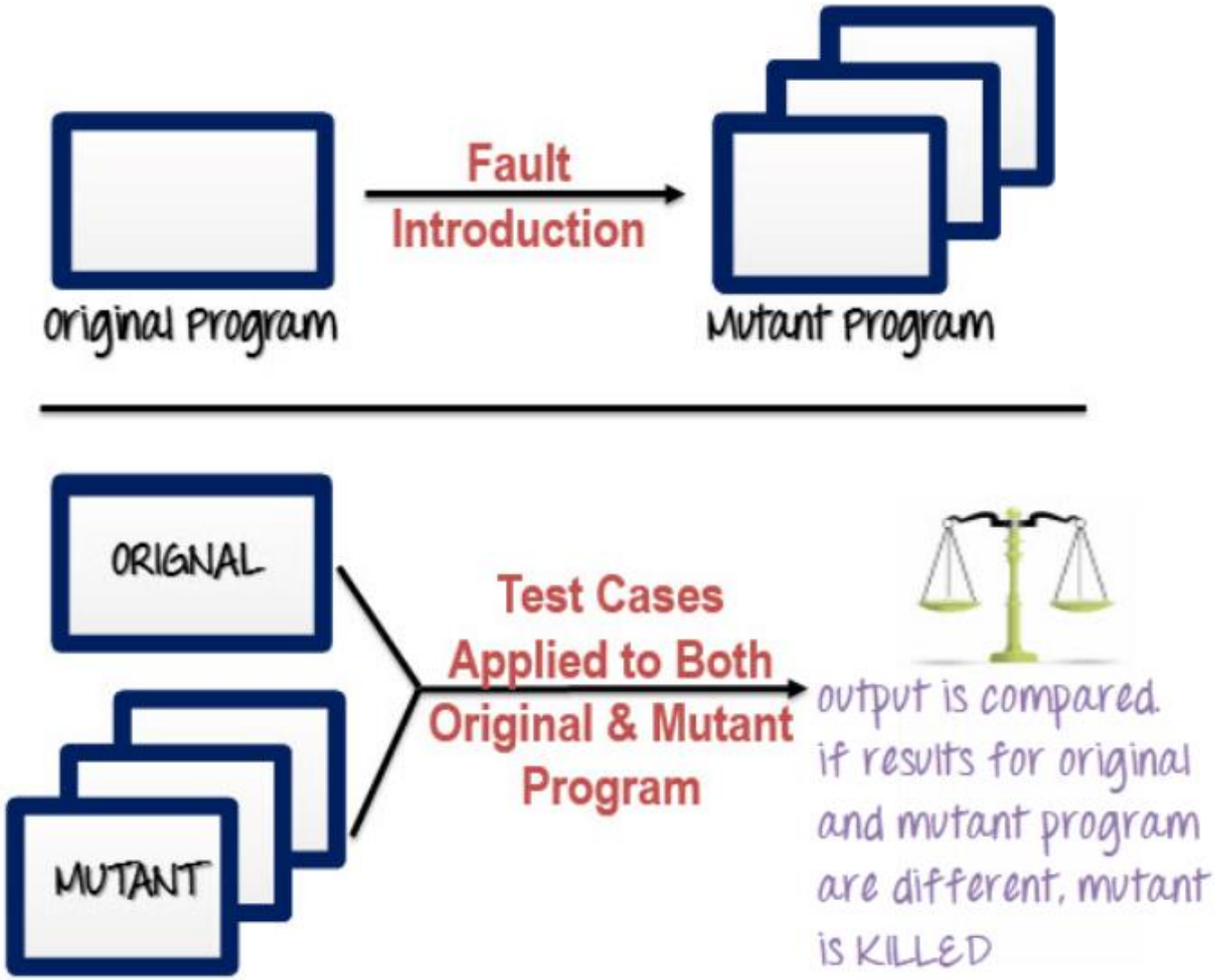
In our earlier equivalence partitioning example, instead of checking one value for each partition, you will check the values at the partitions like 0, 1, 10, 11 and so on. As you may observe, you test values at **both valid and invalid boundaries**. Boundary Value Analysis is also called **range checking**.

Equivalence partitioning and boundary value analysis(BVA) are closely related and can be used together at all levels of testing.

MUTATION TESTING

(evaluate the quality of already existing software tests)

- **Mutation Testing** is a type of software testing in which certain statements of the source code are changed/mutated to check if the test cases are able to find errors in source code. The goal of Mutation Testing is **ensuring the quality of test cases in terms of robustness** that it should fail the mutated source code.
- A mutation of a program is a modification of the program created by introducing a single, small, legal syntactic change in the code. A modified program so obtained is called a mutant.
- A mutant is said to be killed when the execution of a test case causes it to fail and the mutant is considered to be dead.
- Some mutants are equivalent to the given program, that is, such mutants always produce the same output as the original program


Fault
Introduction
original program
Mutant Program
ORIGNAL
MUTANT
Test Cases
Applied to Both
Original & Mutant
Program
output is compared.
if results for original
and mutant program
are different, mutant
is KILLED

If the original program and mutant program generate the same output, Mutant is kept alive. In such cases, more effective test cases need to be created that kill all mutants.

Operand replacement operators	Expression Modification Operators	Statement modification Operators
Replace the operand with another operand (x with y or y with x) or with the constant value.	Replace an operator or insertion of new operators in a program statement.	Programmatic statements are modified to create mutant programs.

Operand replacement operators	Expression Modification Operators	Statement modification Operators
Example- If(x>y) replace x and y values If(5>y) replace x by constant 5	Example- If(x==y) We can replace == into >= and have mutant program as If(x>=y) and inserting ++ in the statement If(x==++y)	Example- Delete the else part in an if-else statement Delete the entire if-else statement to check how a program behaves Some of sample mutation operators: • GOTO label replacement • Return statement replacement • Statement deletion • Unary operator insertion (Like – and ++) • Logical connector replacement • Comparable array name replacement • Removing of else part in the if-else statement • Adding or replacement of operators • Statement replacement by changing the data

- Mutation testing is extremely time consuming and complicated to execute manually. To speed up the process, it is advisable to go for automation tools. Automation tools reduce the cost of testing as well.
- Tool:

Stryker — Javascript

The mutation score is defined as the percentage of killed mutants with the total number of mutants.

- **Mutation Score = (Killed Mutants / Total number of Mutants) * 100**

Advantages of Mutation Testing:

- It brings a good level of error detection in the program.
- It discovers ambiguities in the source code.

Disadvantages of Mutation Testing:

- It is highly costly and time-consuming.